

NLP & MACHINE LEARNING PROJECT

Sentiment Analysis Using NLTK Library

A Comprehensive Guide to Social Media Text Processing & Classification

Venugopalakrishna Vanka

Data Engineer | Data Scientist

Executive Summary & Abstract

Understanding public opinion and customer sentiment in the airline industry through automated NLP

Problem Domain

- The internet and social media platforms generate massive volumes of unfiltered customer feedback daily.
- Manual inspection of feedback is extremely costly, slow, and resource-intensive for major corporations.
- Airlines require automated sentiment analysis to immediately detect operational flaws and public dissatisfaction.

What is Sentiment Analysis?

Computational identification and categorization of opinions expressed in text to determine the writer's attitude toward a specific product, service, or topic (e.g., Positive, Neutral, or Negative).

Solution Overview

- Leveraged Python's **Natural Language Toolkit (NLTK)** and **Scikit-Learn** to build a robust NLP pipeline.
- Preprocessed and vectorized raw Twitter text data to feed machine learning classifiers.
- Evaluated multiple models (Naive Bayes, Random Forest, Logistic Regression) to deliver an optimized sentiment predictor.

Core Tooling Stack

Language: Python 3

NLP Library: NLTK (Tokenization, Regex, Stopwords)

Data Wrangling: Pandas, NumPy

Modeling: Scikit-Learn (TF-IDF, Logistic Regression)

Dataset Architecture & Problem Formulation

Structure and attributes of the US Airline Sentiment dataset

Dataset Characteristics

- **Source:** Kaggle US Twitter Airline Sentiment Dataset.
- **Total Volume:** 14,640 public tweets across 6 major US airlines.
- **Attributes:** 12 features detailing tweet content, timestamps, retweets, and locations.

Problem Statement

Build a supervised multi-class Machine Learning model capable of accurately predicting the sentiment target variable (`airline_sentiment`) using cleaned unstructured tweet text and associated metadata.

Feature Name	Description	Type
<code>airline_sentiment</code>	Target (Negative, Neutral, Positive)	Categorical
<code>text</code>	Raw tweet content string	Text
<code>airline</code>	Targeted airline provider	Categorical
<code>negativereason</code>	Reason for negative feedback	Categorical
<code>airline_sentiment_confidence</code>	Label confidence score	Float
<code>retweet_count</code>	Number of retweets	Integer

Exploratory Data Analysis (EDA)

Key data insights and sentiment distribution patterns

1. Global Sentiment Breakdown

- **Negative Sentiment: 63%** (Predominant majority of user tweets).
- **Neutral Sentiment: 21%** (Inquiries, updates, standard tags).
- **Positive Sentiment: 16%** (Compliments, successful service).

2. Airline Distribution

- **United Airlines:** Highest tweet volume (~26%).
- **US Airways:** Second highest tweet volume (~20%).
- **Virgin America:** Lowest tweet volume (~3%), but maintains the most balanced positive/negative ratio.

3. Sentiment Confidence Correlation

Annotator confidence scores are significantly higher for negative sentiment posts (>0.90) compared to positive and neutral posts (~0.82), indicating strong, unambiguous negative language in complaints.

63%

Negative Feedback

26%

United Airlines
Share

0.92

Negative
Confidence

Data Cleaning & Preprocessing Pipeline

Transforming raw, noisy tweets into standardized clean text

Text Cleaning Operations (Regular Expressions)

- **Special Character Removal:** Strip non-word symbols and punctuation using `re.sub(r'\W', ' ', text)`.
- **Single Character Removal:** Remove standalone letters left by punctuation removal `re.sub(r'\s+[a-zA-Z]\s+', ' ', text)`.
- **Prefix Cleaning:** Strip bytecode artifacts like `b'` prefixes.
- **Whitespace Normalization:** Collapse multiple spaces into a single space.
- **Case Normalization:** Lowercase all tokens for uniformity.

Python Implementation Snippet

```
# Cleaning pipeline execution
processed_features = []
for sentence in range(0, len(features)):
    # Remove special characters
    proc = re.sub(r'\W', ' ',
str(features[sentence]))
    # Remove single characters
    proc = re.sub(r'\s+[a-zA-Z]\s+', ' ', proc)
    # Lowercase & collapse spaces
    proc = re.sub(r'\s+', ' ', proc,
flags=re.I).lower()
    processed_features.append(proc)
```

Feature Extraction & Representation

Converting clean text into high-dimensional numerical vectors

Bag of Words (BoW) Approach

- Constructs a vocabulary of all unique words across the corpus.
- Represents each document as a frequency count vector.
- **Limitation:** Treats all words equally regardless of their informativeness (e.g., "the" vs. "delayed").

TF-IDF Vectorization (Selected)

Term Frequency - Inverse Document Frequency penalizes frequently occurring stop words while assigning higher weights to informative, rare words key to sentiment determination.

TF-IDF Vectorizer Configuration

```
from sklearn.feature_extraction.text import
TfidfVectorizer

vectorizer = TfidfVectorizer(
    max_features=2500,
    min_df=7,
    max_df=0.8,
    stop_words=stopwords.words('english')
)

X =
vectorizer.fit_transform(processed_features).toa
rray()
```

- `max_features=2500`: Top 2,500 informative tokens retained.
- `min_df=7`: Words must appear in at least 7 tweets.
- `max_df=0.8`: Words appearing in >80% of tweets excluded.

Machine Learning Classifiers Evaluated

Supervised algorithms benchmarked for sentiment classification

1. Random Forest

Ensemble learning model combining multiple decision trees to reduce variance and prevent overfitting.

- Handles high-dimensional sparse text vectors effectively.
- Robust against individual noisy features.

2. Multinomial Naive Bayes

Probabilistic classifier assuming strong feature independence given the class label.

- Extremely fast training and inference speeds.
- Baseline standard for NLP text classification tasks.

3. Logistic Regression

Linear classification model optimizing log-odds of target class membership using decision boundaries.

- Exceptional performance on linear high-dimensional TF-IDF space.
- Highly interpretable model coefficients.

Algorithm Performance Comparison

Benchmarking accuracy scores across candidate models

Algorithm	Validation Accuracy	Status
Logistic Regression	77.90%	SELECTED
Random Forest Classifier	75.90%	Evaluated
Multinomial Naive Bayes	75.80%	Evaluated

Train / Test Split Details

- **Train Ratio:** 80% (11,712 samples)
- **Test Ratio:** 20% (2,928 samples)
- **Random State:** 0 (Ensures reproducibility)

Key Performance Insights

- **Logistic Regression** outperformed tree-based and naive probabilistic models by ~2.0% accuracy.
- TF-IDF sparse feature spaces are naturally linearly separable, favoring Logistic Regression's convex optimization objective.
- Multinomial Naive Bayes performed strongly but suffered slightly due to independence assumption violations among correlated words.

Performance Evaluation Metrics

Detailed confusion matrix, precision, and recall analysis

Logistic Regression Confusion Matrix

Actual \ Pred	Negative (A)	Neutral (B)	Positive (C)	Total
Negative (A)	1764	83	23	1870
Neutral (B)	311	261	42	614
Positive (C)	136	50	258	444

Class	Precision	Recall	F1-Score
Negative (A)	0.797	0.943	0.864
Neutral (B)	0.662	0.425	0.518
Positive (C)	0.798	0.581	0.672
Weighted Avg	0.770	0.770	0.763

Class-Level Insights

Negative class achieves a exceptional **94.3% Recall**, making the model highly dependable for capturing customer complaints and operational escalation triggers.

Model Validation & Threshold Tuning

K-Fold cross-validation and decision boundary stability

5-Fold Cross-Validation Framework

- Dataset split into 5 equal folds (Sets A through E).
- Iteratively trained on 4 folds and tested on the remaining fold.
- **Average CV Accuracy Score:** 77.60%.
- **Single Split Test Score:** 77.97%.

Probability Distribution Analysis

- Predicted probability distributions show strong positive skewness toward low positive values.
- Over 800 samples returned probabilities < 0.1 for positive sentiment, reflecting the true negative dominance in airline feedback.

Variance & Stability Analysis

The minimal difference (~0.37%) between cross-validation and single test splits proves the model is highly stable and free from severe variance or train-set overfitting.

77.60%

Stable K-Fold Cross-Validation Accuracy

Conclusion & Business Impact

Summary of achievements, operational utility, and developer links

Project Summary

- Successfully implemented an end-to-end NLP sentiment processing pipeline using Python and NLTK.
- Engineered 2,500 TF-IDF sparse features after regex-based cleaning and noise elimination.
- Achieved **77.9% prediction accuracy** and **94.3% negative class recall** using Logistic Regression.

Business & Industry Applications

- **Automated Escalation:** Automatically route negative feedback to customer support teams in real time.
- **Competitive Benchmarking:** Compare customer satisfaction metrics across competing airlines dynamically.

Developer Profile & Links

Venugopalakrishna Vanka

Data Engineer | Data Scientist

GitHub: github.com/venugopalakrishnavanka

LinkedIn: linkedin.com/in/venugopalakrishnavanka